

Exposé: Anwendung, Vorteile und Derandomisierung von randomisierten Algorithmen

Geis Nico

Fakultät für Informatik

Technische Hochschule Rosenheim

Rosenheim, Deutschland

nico.geis@stud.th-rosenheim.de

I. MOTIVATION

Probabilistische Modelle in der theoretischen Informatik wie z.B. probabilistische Turingmaschinen verwenden Zufall als Teil ihrer Struktur. Sie haben ihre Ursprünge in den 1950er und 1960er Jahren [1] und entwickelten sich seitdem – insbesondere über das letzte Jahrzehnt – von einem Werkzeug der algorithmischen Zahlentheorie [2] zu einem Bestandteil vieler Algorithmen in Bereichen wie der Kryptographie, dem maschinellen Lernen und der Optimierung [3].

Randomisierte Algorithmen können viele Probleme deutlich schneller lösen als ihre deterministischen Gegenstücke. Sie sind in vielen Fällen außerdem einfacher zu beschreiben und zu implementieren als deterministische Algorithmen mit vergleichbarer Leistung. Ein typisches Beispiel für einen Algorithmus, der von der Einbindung zufälliger Entscheidungen profitiert, ist z.B. Randomized Quicksort [2], [4].

Darüber hinaus können randomisierte Algorithmen robuster gegenüber Angriffen sein, bei denen bewusst Worst-Case-Szenarien konstruiert werden [3].

Die Frage, mit der sich diese Seminararbeit beschäftigt, ist: **Was sind randomisierte Algorithmen und welche Vorteile bieten sie gegenüber deterministischen Algorithmen?** In einer kritischen Betrachtung soll außerdem aufgelistet werden, in welchen Anwendungsfällen randomisierte Algorithmen besonders sinnvoll oder eventuell weniger geeignet sind. Dabei können Gegenüberstellungen von randomisierten und deterministischen Algorithmen als Stütze dienen.

Unter Betrachtung möglicher Antwortversuche auf die Frage, ob die Komplexitätsklasse BPP gleich der Klasse P ist, soll außerdem das Konzept der **Derandomisierung** angeschnitten werden, um aufzuzeigen, ob Derandomisierung durch Determinismus ersetzt werden kann und ob sie auch unter der Annahme, dass $P = BPP$ gilt, Vorteile bieten kann.

II. MÖGLICHER AUFBAU

Im Folgenden sind mögliche Inhalte für die Seminararbeit aufgelistet und in eine grobe und vorläufige Reihenfolge gebracht. Je nach Schwerpunktsetzung können und sollten einzelne Punkte vertieft oder weggelassen werden, um die Fragestellung in angemessener Tiefe und innerhalb der Vorgabe von 10 Seiten zu beantworten.

- 1) **Motivation:** Ineffiziente deterministische Algorithmen
 - a) Vorteile von randomisierten Algorithmen (Geschwindigkeit, Einfachheit, Robustheit)
 - b) Beispiele: Primzahltest, randomisierter Quicksort
- 2) **Definition** von randomisierten Algorithmen einschl. **Einteilung** in Monte-Carlo-Algorithmen, Las-Vegas-Algorithmen
- 3) **Komplexitätsklassen** BPP , RP , $coRP$, ZPP
 - a) Erklärung und Zusammenhänge
 - b) Frage: $P = BPP$?
- 4) **Derandomisierung**
 - a) Methoden: Pseudorandom-Generatoren, Expander-Graphen
 - b) Beispiel: Derandomisierung an einem Beispiel (z.B. Min-Cut-Algorithmus)
- 5) **Kritische Betrachtung:** Wann sind randomisierte Algorithmen (nicht) sinnvoll? (Anwendungsfälle, Grenzen)

A. Motivation

Die Motivation lässt sich gut mit Beispielen von Problemen begründen, die bekannte deterministische Algorithmen nicht effizient lösen können, während randomisierte Algorithmen deutlich schneller sind. Ein Beispiel dafür ist die Primzahlprüfung, die mit deterministischen Algorithmen wie dem AKS-Primzahltest in polynomieller Zeit gelöst werden kann, aber in der Praxis oft zu langsam ist. Der probabilistische Solovay-Strassen-Test ist dagegen sehr schnell [5].

Der Zufallsaspekt des randomisierten Quicksort-Algorithmus soll die Wahrscheinlichkeit des Worst-Case-Szenarios reduzieren, bei dem die Sortierung einer bereits sortierten Liste eine Komplexität von $\mathcal{O}(n^2)$ aufweist, um den Algorithmus so robuster machen [6].

B. Definition, Einteilung und Komplexitätsklassen

Randomisierte Algorithmen verwenden Zufall als Ressource [2]. Woher die Zufallszahlen kommen, die von randomisierten Algorithmen verwendet werden, ist eine wichtige und nicht-triviale Frage, die in der Seminararbeit angesprochen werden soll. In der Definition sollen die verschiedenen Arten von randomisierten Algorithmen wie Monte-Carlo- und Las-Vegas-Algorithmen unterschieden werden. Diese weisen unterschiedliche Fehlerwahrscheinlichkeiten und

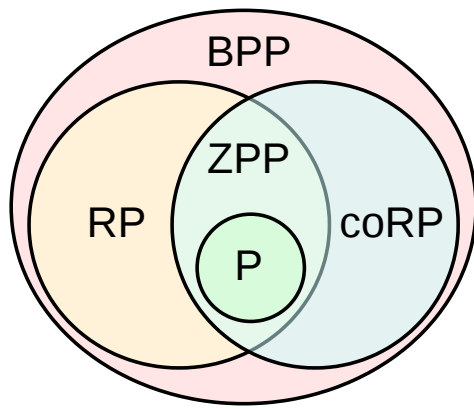


Abbildung 1. Zusammenhang zwischen den Komplexitätsklassen P , RP , $coRP$, ZPP und BPP . Quelle: Eigene Darstellung.

Laufzeitgarantien auf [2]. An dieser Stelle kann auch das Konzept der Amplifikation angesprochen werden, um zu zeigen, wie sich das wiederholte Ausführen eines randomisierten Algorithmus auf die Fehlerwahrscheinlichkeit auswirkt. Darüber hinaus kann bei Bedarf der Unterschied zwischen zufälligem Input, zufälligem Algorithmus und Heuristiken herausgearbeitet werden, um klarzustellen, was (nicht) als randomisierter Algorithmus verstanden wird.

Die beiden Arten von randomisierten Algorithmen, Monte-Carlo- und Las-Vegas-Algorithmen, liegen in unterschiedlichen Komplexitätsklassen, die in der Seminararbeit vorgestellt werden sollen. Las-Vegas-Algorithmen liegen in der Klasse ZPP (Zero-error Probabilistic Polynomial time) und liefern immer die korrekte Lösung, aber ihre Laufzeit ist nur im Erwartungswert polynomiell.

Monte-Carlo-Algorithmen weisen dagegen eine deterministisch begrenzte Laufzeit auf. Sie lassen sich weiter unterteilen, je nachdem ob Entscheidungsprobleme zweiseitig oder einseitig fehlerhaft gelöst werden können. Ein *zweiseitiger Fehler* bedeutet, dass der Algorithmus mit einer gewissen Fehlerwahrscheinlichkeit sowohl positive Antworten auf falsche Instanzen als auch negative Antworten auf richtige Instanzen liefern kann. Solche Algorithmen liegen in der Klasse BPP (Bounded-error Probabilistic Polynomial time). Ein *einseitiger Fehler* bedeutet, dass der Algorithmus nur in eine Richtung Fehler machen kann. Je nach Richtung des Fehlers liegen solche Algorithmen in einer von zwei komplementären Komplexitätsklassen, die beide innerhalb von BPP liegen. In der Klasse RP (Randomized Polynomial time) kann eine falsche Instanz fälschlicherweise akzeptiert werden, aber eine richtige Instanz wird immer korrekt akzeptiert. Das Verhalten in der Klasse $coRP$ (complement of RP) ist genau umgekehrt.

Die Schnittmenge der Klassen RP und $coRP$ bildet die Klasse ZPP , in der die Klasse P liegt. Abbildung 1 zeigt die Beziehung zwischen diesen Klassen. Die Vermutung, dass P gleich BPP ist, ist noch eine der offenen Fragen in der

Komplexitätstheorie.

C. Frage $P = BPP?$ und Derandomisierung

Alle Probleme in P liegen auch in BPP , da deterministische Algorithmen als Spezialfälle von randomisierten Algorithmen betrachtet werden können, die keine Zufallsentscheidungen treffen. Doch da es Probleme in BPP gibt, von denen nicht bekannt ist, ob sie auch in P liegen, ist die Frage, ob P gleich BPP ist, noch offen.

Eine Herangehensweise zur Beantwortung dieser Frage ist die Untersuchung von Methoden der Derandomisierung, die darauf abzielen, randomisierte Algorithmen durch deterministische Algorithmen zu ersetzen, ohne dabei die Effizienz signifikant zu beeinträchtigen. Derandomisierung kann über verschiedene Techniken erreicht werden, die teilweise nur unter speziellen Bedingungen funktionieren oder allgemein zu ineffizient sein können. Typische Werkzeuge zur Derandomisierung sind z. B. Pseudorandom-Generatoren und Expander-Graphen [7].

Allgemein ist unklar, ob es möglich ist, alle randomisierten Algorithmen effizient zu derandomisieren. Falls ein allgemeingültiger Algorithmus zur Derandomisierung existieren würde, könnte dies $P = BPP$ bedeuten, da alle Probleme, die mit randomisierten Algorithmen gelöst werden können, ohne signifikanten Effizienzverlust auch mit deterministischen Algorithmen gelöst werden könnten.

Impagliazzo und Wigderson [8] vertreten die Vermutung, dass die Klassen P und BPP unter weiteren Annahmen vermutlich gleich sind. Weitere Literatur zu diesem Thema findet sich z. B. in [9] und [10].

D. Anwendungsfälle und Grenzen von randomisierten Algorithmen

In der Seminararbeit soll die Frage beantwortet werden, für welche Anwendungsfällen randomisierte Algorithmen besonders gut geeignet sind. Die genannten Vorteile von randomisierten Algorithmen wie Geschwindigkeit, Einfachheit und Robustheit können hier durch Gegenüberstellungen deterministischer und randomisierter Algorithmen genauer betrachtet werden. Vielleicht kann dabei sogar hinterfragt werden, ob diese Vorteile auch unter $P = BPP$ fortbestehen würden.

Karp [11] zeigt bereits die verschiedenen Anwendungsmöglichkeiten randomisierter Algorithmen in Bereichen wie z. B. der Zahlentheorie, der Algebra, der Graphentheorie und der Kombinatorik auf. Die wichtige Rolle von randomisierten Algorithmen in der Kryptographie, z. B. bei probabilistischen Verschlüsselungen wie von Goldwasser und Micali [12] beschrieben, ist an dieser Stelle aufzuzeigen, um zu verdeutlichen, dass randomisierte Algorithmen auch in der Praxis Anwendung finden.

Ein kritischer Blick auf die Anwendung in sicherheitskritischen Systemen, bei denen Fehler fatale Auswirkungen haben können, kann hier ebenfalls beleuchtet werden, da randomisierte (Monte-Carlo-)Algorithmen mit einer gewissen Fehlerwahrscheinlichkeit arbeiten. Sichere probabilistische Techni-

ken werden beispielsweise in [13] untersucht. Als Schwierigkeit bei der Verwendung von randomisierten Algorithmen kann die Bereitstellung von Zufallszahlen genannt werden, da echte Zufallszahlen schwer bzw. aufwendig zu generieren sind.

LITERATUR

- [1] M. Antonelli, U. D. Lago, and P. Pistone, “On randomized computational models and complexity classes: a historical overview,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.11999>
- [2] R. Motwani and P. Raghavan, *Randomized Algorithms*, ser. Cambridge International Series on Parallel Computation. Cambridge: Cambridge University Press, 1995, illustrated edition; also part of the Stanford-Cambridge program.
- [3] T. Recruiter, “Randomized algorithms: Using chance for computational success,” Substack, Dec. 2024, accessed: 2026-04-17. [Online]. Available: <https://techrecruiter.substack.com/p/randomized-algorithms-using-chance>
- [4] Saurabh, “Randomized algorithms: A new era of problem solving with uncertainty,” Medium article, Apr. 2025, accessed: 2026-04-17. [Online]. Available: <https://medium.com/@kadamsaurabh5601/randomized-algorithms-a-new-era-of-problem-solving-with-uncertainty-b8e51aaa9dbe>
- [5] R. Solovay and V. Strassen, “A fast monte-carlo test for primality,” *SIAM Journal on Computing*, vol. 6, no. 1, pp. 84–85, 1977. [Online]. Available: <https://doi.org/10.1137/0206006>
- [6] Tutorialspoint, “Randomized quick sort algorithm,” accessed: 2026-04-17. [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/dsa_randomized_quick_sort_algorithm.htm
- [7] S. P. Vadhan, “Pseudorandomness,” *Foundations and Trends in Theoretical Computer Science*, vol. 7, no. 1–3, pp. 1–336, 2012.
- [8] R. Impagliazzo and A. Wigderson, “ $P = BPP$ if e requires exponential circuits: derandomizing the xor lemma,” in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, ser. STOC '97. New York, NY, USA: Association for Computing Machinery, 1997, p. 220–229. [Online]. Available: <https://doi.org/10.1145/258533.258590>
- [9] O. Goldreich, *In a World of $P=BPP$* . Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 191–232. [Online]. Available: https://doi.org/10.1007/978-3-642-22670-0_20
- [10] R. Impagliazzo, “Hardness as randomness: a survey of universal derandomization,” 2003. [Online]. Available: <https://arxiv.org/abs/cs/0304040>
- [11] R. M. Karp, “An introduction to randomized algorithms,” *Discrete Applied Mathematics*, vol. 34, no. 1, pp. 165–201, 1991. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0166218X9190086C>
- [12] S. Goldwasser and S. Micali, “Probabilistic encryption,” *Journal of Computer and System Sciences*, vol. 28, no. 2, pp. 270–299, 1984. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/002200084900709>
- [13] K. Agrawal, S. Baruah, Z. Guo, and J. Li, “The safe and effective application of probabilistic techniques in safety-critical systems,” in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.